

Machine Learning

CSE 343/543

Lecture 6

Classification – Naïve Bayes, k-NN

Today's Lecture

- Classification
 - Probabilistic classification (Bayes)
 - Nonparametric classification (KNNs or K-nearest neighbors)
 - Non-metric classification (Decision Trees)

Linear Regression:

Linear Map + zero-mean Gaussian errors

- Probabilistic Model: $y \sim \mathcal{N}(f(\mathbf{x}^i, \mathbf{w}), \sigma^2)$
- Parameters – $\mathbf{w}$

$$\mathcal{L}(y, f(\mathbf{x}, \mathbf{w})) = \sum_{i=1}^N L(f(\mathbf{x}^i, \mathbf{w}), y^i) + \lambda R(\mathbf{w})$$

$$\hat{y}^i = f(\mathbf{x}^i, \mathbf{w}) = \mathbf{w}^\top \mathbf{x}^i$$

$$\hat{y}^i = f(\mathbf{x}^i, \mathbf{w}) = w_0 x_0^i + w_1 x_1^i + w_2 x_2^i + \dots$$

x_j^i : j^{th} element of the i^{th} sample vector $\mathbf{x}^i$
Usually, $x_0^i = 1$

- Squared loss

$$L(f(\mathbf{x}^i, \mathbf{w}), y^i) = (y^i - f(\mathbf{x}^i, \mathbf{w}))^2$$

Regularization

$$L(\mathbf{w}) = -\ell(\mathbf{w}) + \lambda R(\mathbf{w})$$

- Gaussian prior or L_2 regularizer or Ridge Regression

$$R(\mathbf{w}) = ||\mathbf{w}||_2^2$$

- Laplacian prior or L_1 regularizer or LASSO (Least absolute shrinkage and selection operator)

$$R(\mathbf{w}) = ||\mathbf{w}||_1$$

- Gaussian+Laplacian Mixture Prior or Elastic-net

$$R(\mathbf{w}) = ||\mathbf{w}||_2^2 + \lambda_1 ||\mathbf{w}||_1$$

Logistic Regression: Linear Map + logit + Bernoulli model

- Bernoulli Model

$$\begin{aligned} Pr(y|x; p) &= \begin{cases} p, & y = 1 \\ 1 - p, & y = 0 \end{cases} \\ &= p^y (1 - p)^{(1-y)} \end{aligned}$$

- Log-Likelihood function

$$\ell(\mathbf{w}) = \sum_{i=1}^N y^i \log p(\mathbf{x}^i; \mathbf{w}) + (1 - y^i) \log (1 - p(\mathbf{x}^i; \mathbf{w}))$$

- Loss function

- also happens to be the binary cross-entropy loss (for $\lambda = 0$)

$$L(\mathbf{w}) = -\ell(\mathbf{w}) + \lambda R(\mathbf{w})$$

$$\begin{aligned} p(\mathbf{x}^i; \mathbf{w}) \\ &= \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x})} \end{aligned}$$

Probabilistic Classifier: Classification Rule

- Classification is then performed as

- If $p(\hat{y}^i = 1 | \mathbf{x}^i; \mathbf{w}) > p(\hat{y}^i = 0 | \mathbf{x}^i; \mathbf{w})$

then the label

$$\hat{y}^i = 1$$

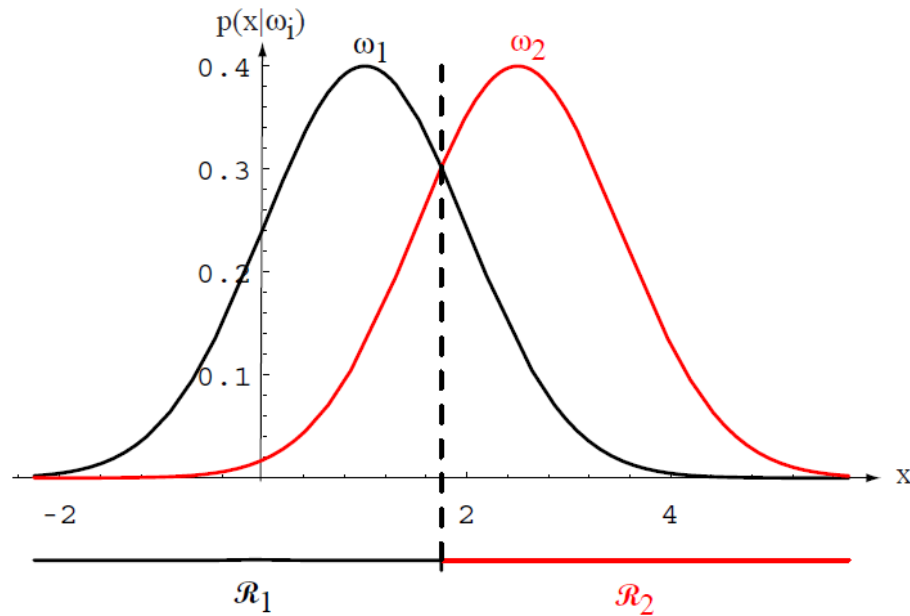
else

$$\hat{y}^i = 0$$

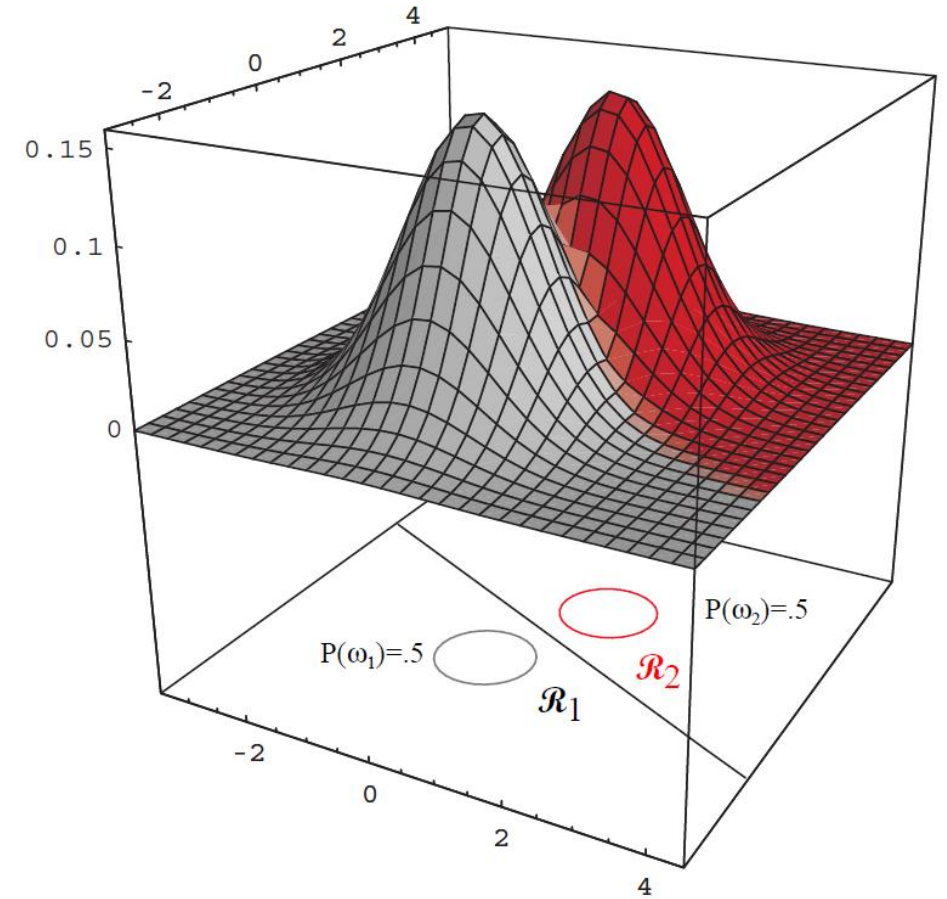
- For M-class classification

$$\hat{y}^i = \arg \max_{y=\{0,1,\dots,M\}} p(f(\mathbf{x}^i, \mathbf{w}) = y | \mathbf{x}^i; \mathbf{w})$$

Probabilistic Classifier: Decision Boundary Visualization



1-D



2-D

Discriminative vs. Generative Probabilistic Classifier

- For Binary Classification, we are interested in predicting $y \in \{0,1\}$
 - Consequently in $p(y|\mathbf{x})$

- A discriminative classifier **directly models** $p(y|\mathbf{x})$
 - Logistic Regression:

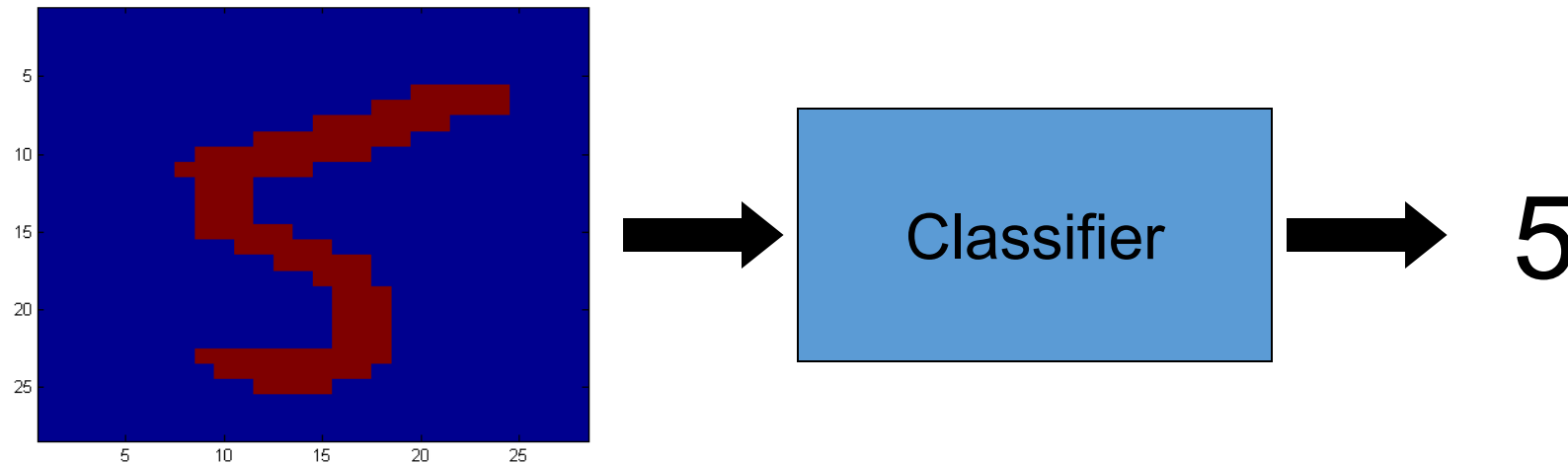
$$p(y = 1|\mathbf{x}; \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x})}$$

- A generative classifier models $p(y|\mathbf{x})$ via the **input data distribution, given a specific class**
 - Applying Bayes Rule

$$p(y|\mathbf{x}) = \frac{p(\mathbf{x}|y)p(y)}{p(\mathbf{x})}$$

Concrete Example: Generative Classifier

- **Digit Recognition**



- $X_1, \dots, X_n \in \{0,1\}$ (Black vs. White pixels)
- $Y \in \{5,6\}$ (predict whether a digit is a 5 or a 6)

The Bayes Classifier

- We saw that a good strategy is to predict:

$$\arg \max_Y P(Y|X_1, \dots, X_n)$$

- (for example: what is the probability that the image represents a 5 given its pixels?)
- So ... How do we compute that?

The Bayes Classifier

- Uses Bayes Rule!

$$\begin{array}{c} \text{Posterior} \\ \downarrow \\ P(Y|X_1, \dots, X_n) \end{array} = \frac{\begin{array}{c} \text{Likelihood} \\ \downarrow \\ P(X_1, \dots, X_n|Y) \end{array} \begin{array}{c} \text{Prior} \\ \downarrow \\ P(Y) \end{array}}{\begin{array}{c} \uparrow \\ \text{Normalization Constant} \\ P(X_1, \dots, X_n) \end{array}}$$

- Why did this help?
 - Well... We think that we might be able to specify how features are “generated” by the class label

The Bayes Classifier

- Let's expand this for our digit recognition task:

$$P(Y = 5|X_1, \dots, X_n) = \frac{P(X_1, \dots, X_n|Y = 5)P(Y = 5)}{P(X_1, \dots, X_n|Y = 5)P(Y = 5) + P(X_1, \dots, X_n|Y = 6)P(Y = 6)}$$
$$P(Y = 6|X_1, \dots, X_n) = \frac{P(X_1, \dots, X_n|Y = 6)P(Y = 6)}{P(X_1, \dots, X_n|Y = 5)P(Y = 5) + P(X_1, \dots, X_n|Y = 6)P(Y = 6)}$$

- To classify, we'll simply compute these two probabilities and predict based on which one is greater

Model Parameters

- For the Bayes classifier, we need to “learn” two functions, the likelihood and the prior
- How many parameters are required to specify the prior for our digit recognition example?

Model Parameters

- How many parameters are required to specify the likelihood?
 - (Supposing that each image is 30x30 pixels)
- # of parameters for modeling $P(X_1, \dots, X_n | Y)$:
 - $2(2^n - 1)$
- The problem with explicitly modeling $P(X_1, \dots, X_n | Y)$ is that there are usually way too many parameters:
 - We'll run out of space
 - We'll run out of time
 - And we'll need tons of training data (which is usually not available)

x_1	x_2
x_3	x_4

The joint distribution of (x_1, x_2, x_3, x_4) , given $Y=1$ is given by assigning probabilities to each of the possible inputs.

$$x_i \in \{0, 1\}$$

$$i=1, 2, 3, 4$$

$$\vec{x} = (x_1, x_2, x_3, x_4)$$

x_1	x_2	x_3	x_4	$P(x_1, x_2, x_3, x_4 Y=1)$
0	0	0	0	p_1
0	0	0	1	p_2
0	0	1	0	p_3
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\vdots$	1	1	0	p_{15}
1	1	1	1	$1 - \sum_{i=1}^{15} p_i$

Total # parameters needed to define the likelihood

$$P(x_1, x_2, x_3, x_4 | Y=1)$$

$$= 15$$

$$= 2^4 - 1$$

$$= 2^n - 1 \Big|_{n=4}$$

∴ for a 30×30 image, pixels are

$$x_1, x_2, \dots, x_n \Big|_{n=900} \Rightarrow \# \text{ params} = 2^n - 1 \text{ for } P(\vec{x} | Y=1)$$

The Naïve Bayes Model

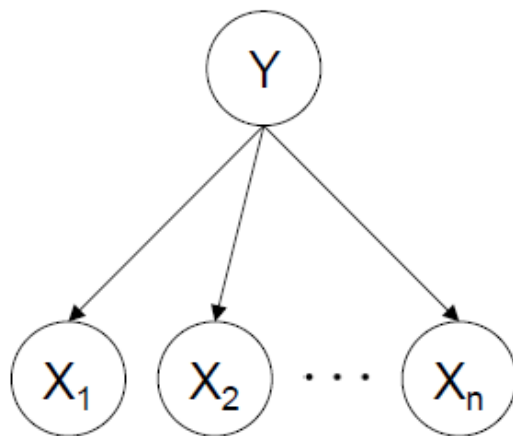
- The *Naïve Bayes Assumption*: Assume that all features are independent **given the class label Y**
- Equationally speaking:

$$P(X_1, \dots, X_n | Y) = \prod_{i=1}^n P(X_i | Y)$$

- # of parameters for modeling $P(X_1 | Y), \dots, P(X_n | Y)$
 - $2n$

The Naïve Bayes Classifier

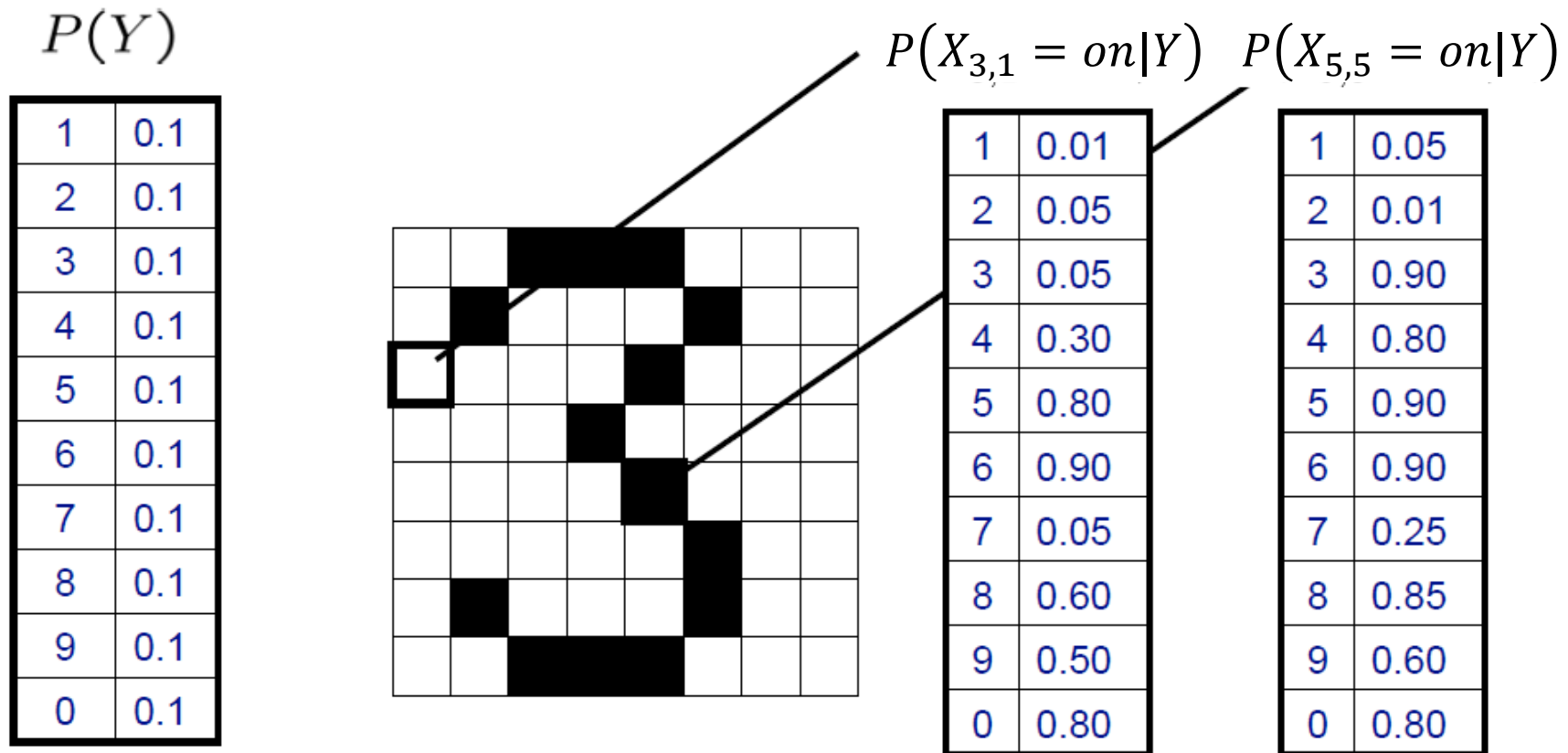
- Given:
 - Prior $P(Y)$
 - n conditionally independent features $\mathbf{X}$ given the class Y
 - For each X_i , we have likelihood $P(X_i | Y)$



- Decision rule:

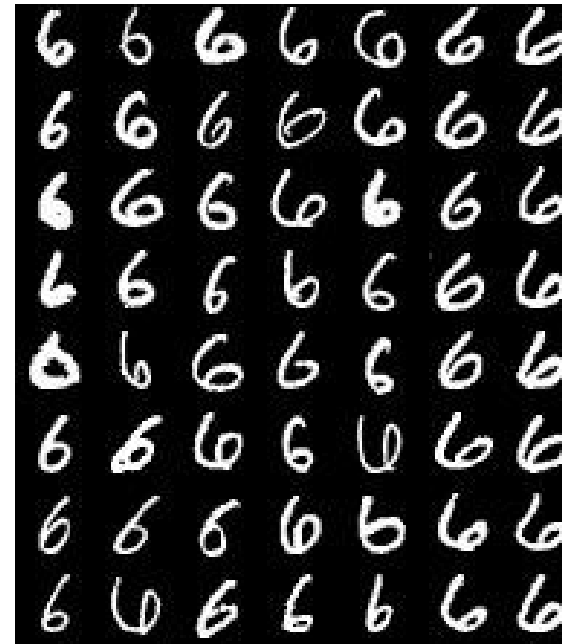
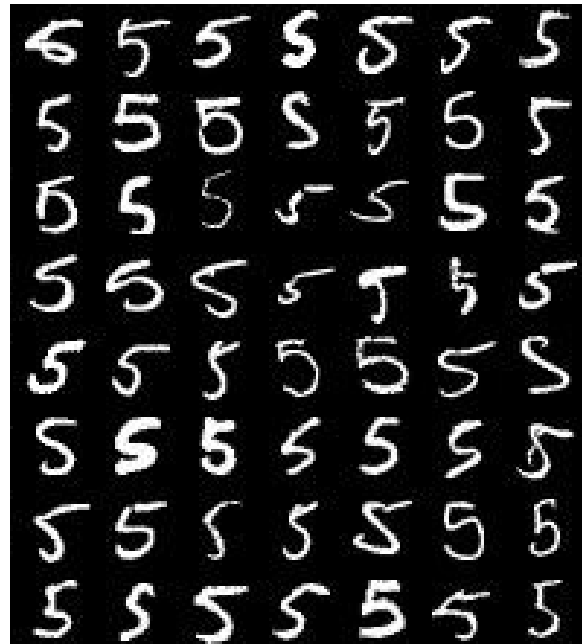
$$\begin{aligned}\hat{y} &= \arg \max_y P(y) P(x_1, \dots, x_n | y) \\ &= \arg \max_y P(y) \prod_i P(x_i | y)\end{aligned}$$

What has to be learned?



Naïve Bayes Training

- Now that we've decided to use a Naïve Bayes classifier, we need to train it with some data:



MNIST Training Data

Naïve Bayes Training

- Training in Naïve Bayes is **easy**:
 - Estimate $P(Y=v)$ as the fraction of records with $Y=v$

$$P(Y = v) = \frac{\text{Count}(Y = v)}{\# \text{ records}}$$

- Estimate $P(X_i=u | Y=v)$ as the fraction of records with $Y=v$ for which $X_i=u$

$$P(X_i = u | Y = v) = \frac{\text{Count}(X_i = u \wedge Y = v)}{\text{Count}(Y = v)}$$

- (This corresponds to Maximum Likelihood estimation of model parameters)

Naïve Bayes Training

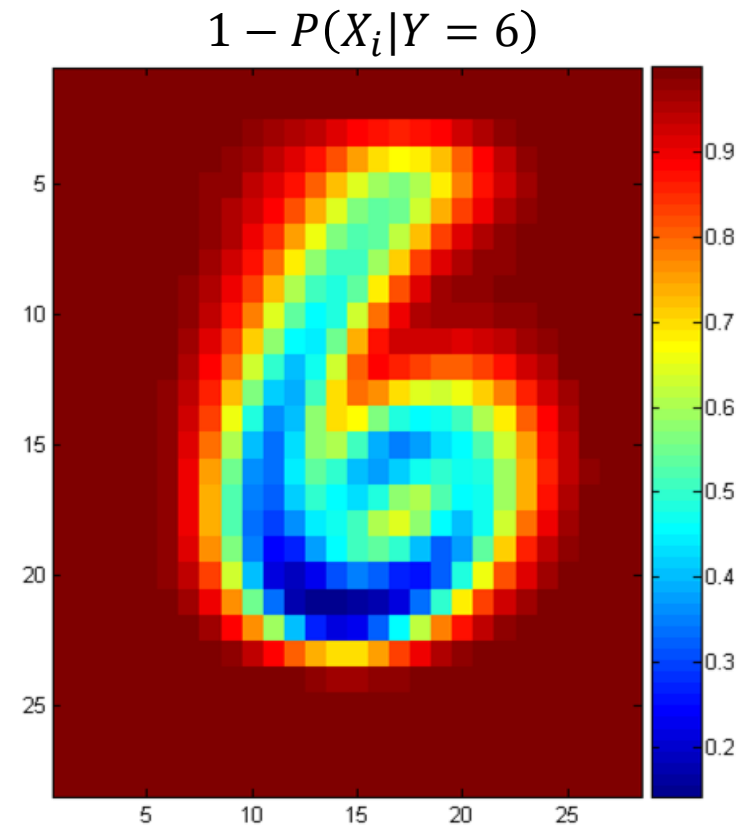
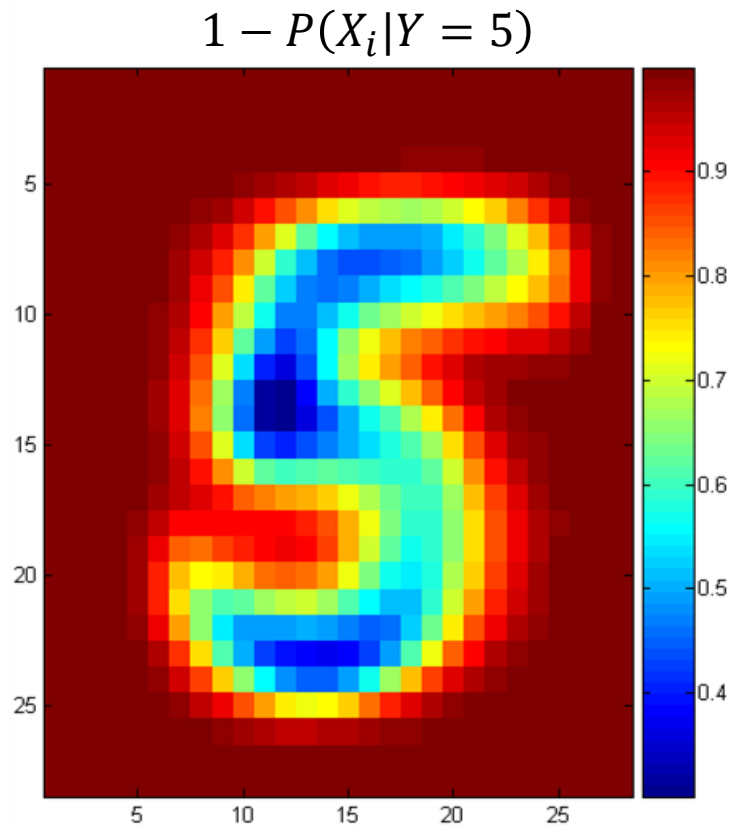
- In practice, some of these counts can be zero
- Fix this by adding “virtual” counts:

$$P(X_i = u|Y = v) = \frac{\text{Count}(X_i = u \wedge Y = v) + 1}{\text{Count}(Y = v) + 2}$$

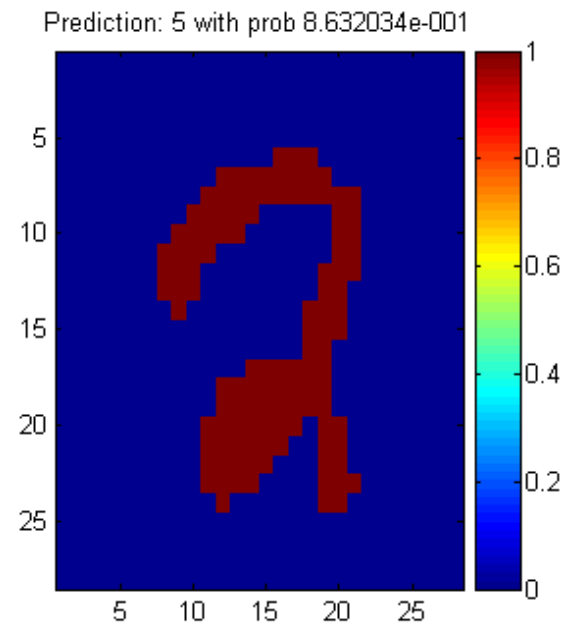
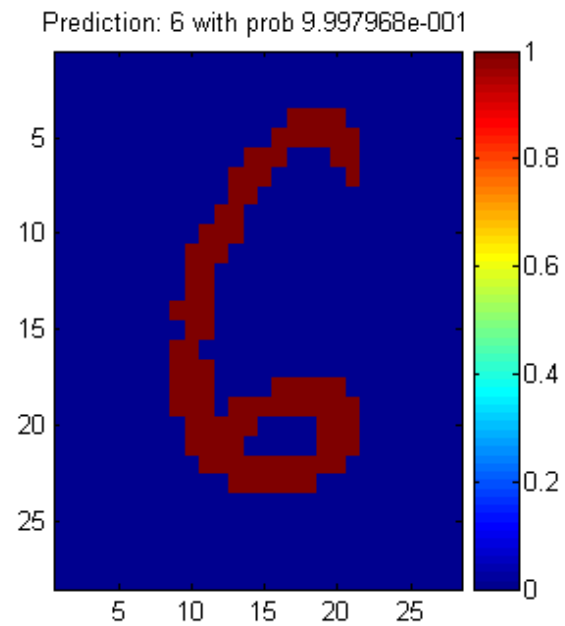
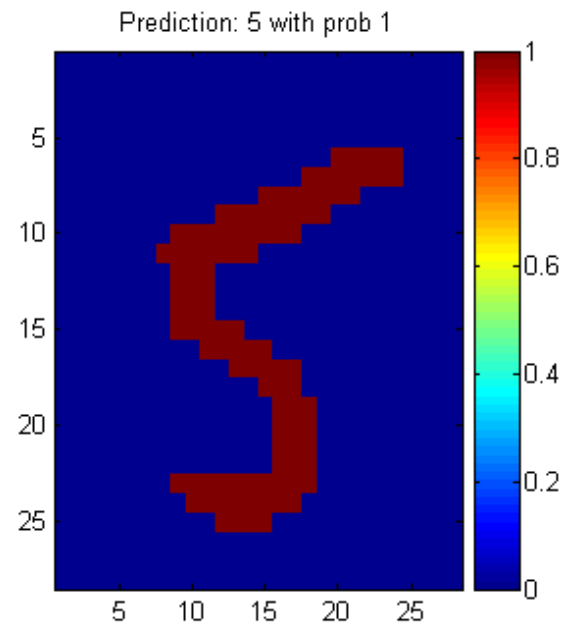
- (This is like putting a prior on parameters and doing MAP estimation instead of MLE)
- This is called *Smoothing*

Naïve Bayes Training

- For binary digits, training amounts to averaging all of the training fives together and all of the training sixes together.

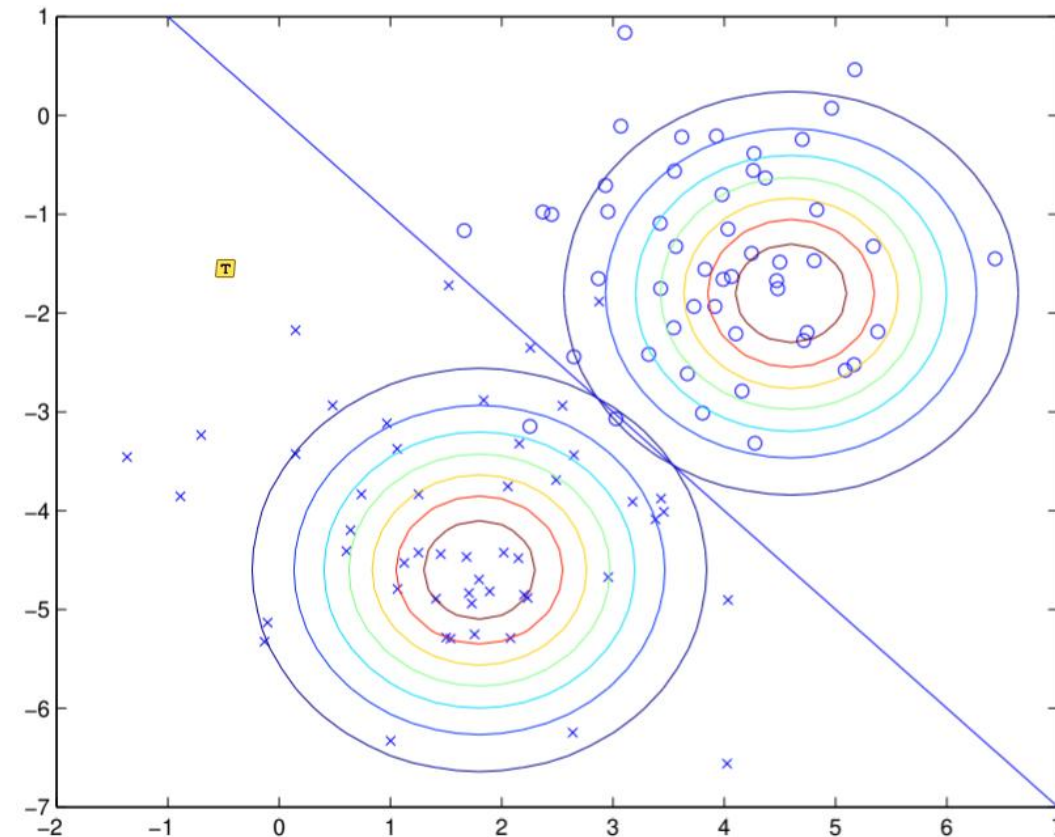
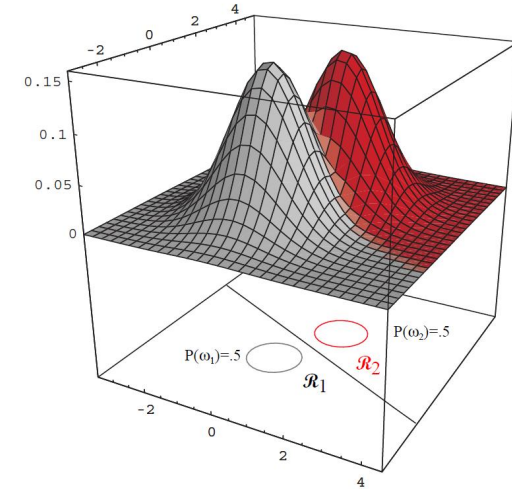


Naïve Bayes Classification



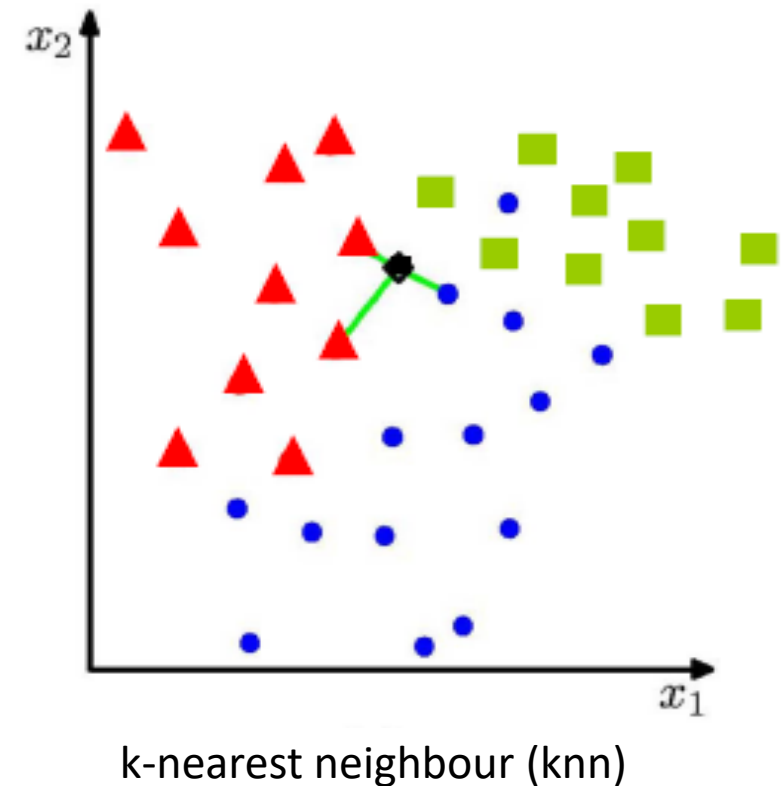
Gaussian Discriminant Analysis (GDA)

- What if our input data follows a Gaussian Distribution, i.e.,
 - $p(\mathbf{x}|y = 0) = \mathcal{N}(\mu_0, \Sigma)$
 - $p(\mathbf{x}|y = 1) = \mathcal{N}(\mu_1, \Sigma)$
 - $y \sim \text{Bernoulli}(\phi)$
- We get what is called as the Gaussian Discriminant Analysis model
 - As in other cases, we can compute the MLE estimates of its parameters

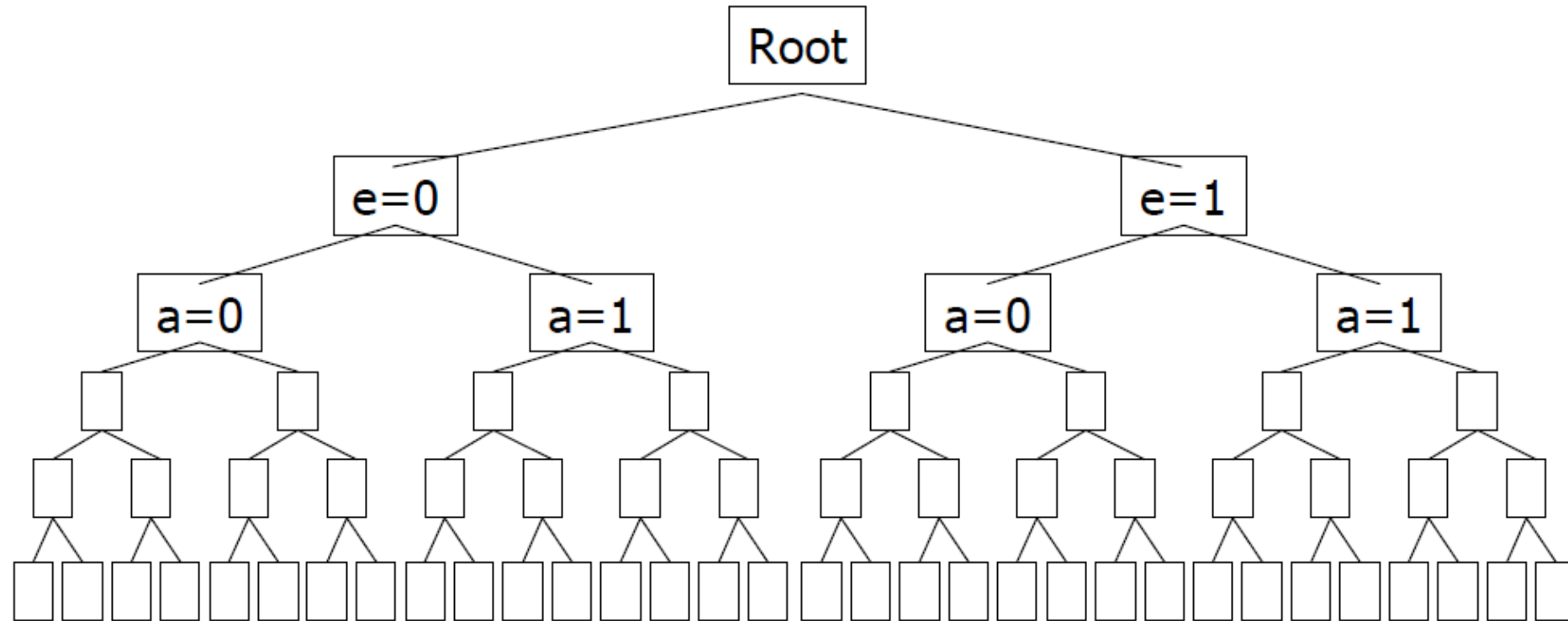


Non-parametric Classification: K-Nearest Neighbors

- Collect training data (x,y)
- For test sample
 - Find nearest neighbors in training set
 - Assign class label based on some consensus
- Parameters
 - Distance Metric for 'nearest' neighbors
 - Voting strategy



Non-metric Classification: Decision Trees



Next Class

- Unsupervised Learning